

Loading Data



Learn2 CloudData

Powered by

www.learn2clouddata.com

Sujith Nair

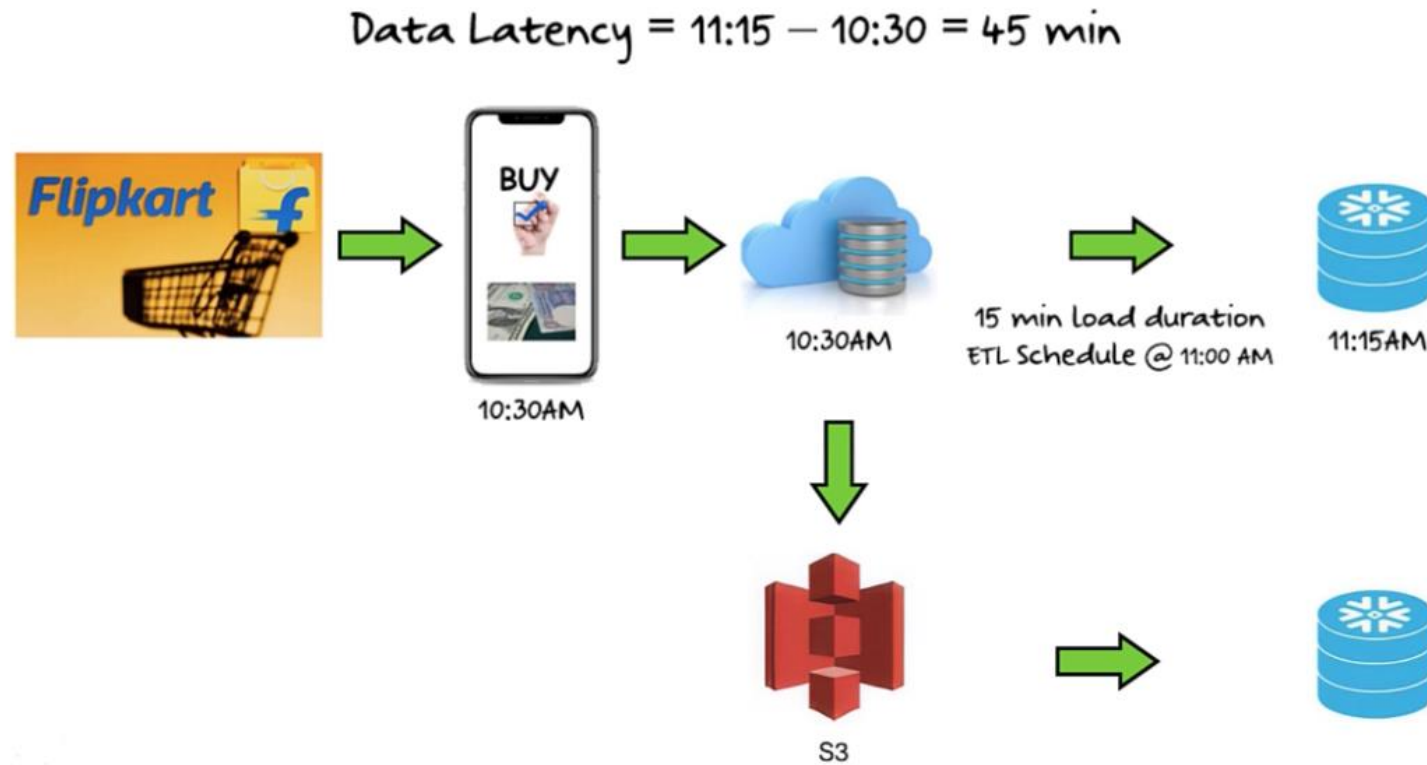
Cloud Data Architect
Snowflake Snowpro Certified

Section Introduction-Loading Data to Snowflake

- What is Data Latency ?
- What are the Loading Types supported in Snowflake?
- Understanding the COPY statement.
- What are COPY options ?
- LAB-Loading Data to Snowflake.
 - Basic COPY to target table.
 - COPY option **FORCE**.
 - Loading a subset of columns with COPY.
 - Copy option **FILES**.
 - Copy option **PATTERN**.
 - Copy option **TRUNCATECOLUMNS**.
 - COPY option **ON_ERROR**.
 - COPY option **RETURN_FAILED_ONLY**.
 - COPY option **SIZE_LIMIT**.
 - COPY option **VALIDATION_MODE**.
 - Loading data to Snowflake overriding the file format.
 - Loading data from s3 to Snowflake without a stage.
 - Transforming data in the COPY statement.
 - Re-processing failed rows in the COPY statement.
 - Loading data to Snowflake using the Snowsight Web UI Wizard.
 - Summary

What is Data latency ?

Data latency is the time difference between data changing in the source system and these changes being made available in the target tables or data warehouse.



What are the loading types supported in Snowflake ?

Data loading in Snowflake can be done in 3 ways

- Bulk Loading.
- Near Real Time Loading.
- Real Time Loading.

Bulk Loading: Bulk loading of data is the process of loading source system data file in batches from cloud storage to tables in Snowflake at regular intervals of time.

Features of Bulk Loading:

- Setup a scheduler to run the COPY on daily, weekly, hourly or any other schedule basis.
- Data Latency is generally more than 15 mins.
- Bulk loading is the most common type of loading of data files to Snowflake tables.
- Bulk loading uses COPY statement to load data from s3 to Snowflake tables.

What are the loading types supported in Snowflake ?

Advantages of Bulk Loading

- Simplicity
 - Fixed Interval Extraction.
 - Easy to fix failure and reload.
 - Easy to understand and implement. Low Complexity.
- Lower Cost
 - Near Real time and Real Time loading of Data consume more credits than Bulk Loading of data due to more usage of credits to maintain smaller latency.
 - Bulk loading requires less human support as compared to Near Real Time and Real Time loading due to less strict SLA for fixes. This reduces labor cost.
- Supports multiple file formats
 - Bulk loading with COPY supports JSON,AVRO,ORC,PARQUET,XML and CSV.
 - Real Time data ingestion support only JSON and AVRO.
- Customization
 - Bulk loading supports several COPY options to change default loading behavior like rejecting files if the number of errors are over a certain limit.
 - Near Real Time and Real Time data load do not have some of these loading options.

What are the loading types supported in Snowflake ?

Near Real Time Loading: Near real-time loading of data refers to the process of frequently ingesting data into tables with low latency which is generally a few mins.

Features of Near Real Time Loading:

- There is no scheduling for Near Real Time data load, it is event driven and on the occurrence of the event which is the arrival of the file on s3, the file event is processed and loaded.
- Near real time data is ingested within a few mins of the arrival of the data whereas there is no such requirement for bulk loads.
- Near Real time loading is done with Snowpipe in Snowflake.
- Files processed in near real time loads tend to be smaller than those used in bulk loads.

Advantages of Near Real Time Loading:

- Data is made available in target tables very quickly within a few mins.
- No need to spend time scheduling loads saving data engineer time.
- Saving of credits in those cases when the files appear intermittently or randomly.

What are the loading types supported in Snowflake ?

Real Time Loading: Real-time loading of data involves continuously ingesting data into snowflake tables immediately as it is created or updated in the source systems, with negligible latency.

Real Time loading of data ensures that the data is available for querying and analysis almost instantaneously after any changes occur in the source. The latency for Real Time data is just a few seconds.

Features of Real Time Loading:

- There is no scheduling for Real Time data load, data is streamed continuously from the source and ingested into Snowflake tables.
- Real time loading of data is done with the help of Apache Kafka and using a Snowflake connector for Apache Kafka.

Advantages of Real Time Loading:

- Real time data is ingested within a few seconds of generation of the data in the source.

Disadvantage of Real Time Loading:

- Higher cost as compared to Bulk loading and Near Real time loading.
- Limited support for formats. Support for Avro and JSON is provided. other formats are not supported.

Understanding the COPY statement.

The COPY statement is designed to load data from Snowflake stages(External or Internal) to tables in Snowflake. It can also be used to unload data from Snowflake tables to stages on AWS or internal stages.

```
COPY INTO EMPLOYEES  
FROM @HRMS.HR. AWS_ETL_CSV_STAGE.
```

Features of COPY statement:

- COPY statement loads all the files inside the stage to the target table.
- There is no need to provide the file format as the File Format was provided when we created the Stage.
- By default any file that is loaded to a target table will not be re-loaded even if the COPY statement is re-run.
- Default COPY behavior can be changed using COPY options.
- COPY statement matches fields in the file and the table by position for loading.

What are COPY options ?

A COPY option is an instruction that is added to the COPY statement to alter the default behavior of the COPY statement.

For Eg: If you have a file with 100,000 rows and it has 1 bad row in the file ,the load will be aborted when it encounters the error. The reason this happens is because the default action when the COPY encounters an error is to abort the transaction.

```
COPY INTO EMPLOYEES  
FROM @HRMS.HR. AWS_LOAD_CSV_STAGE  
ON_ERROR=CONTINUE
```

Use COPY option ON_ERROR and set it to CONTINUE which will suppress the default behavior and load the good rows in the file to the target table. It will reject only the bad rows in the file

Summary

- Data latency is the time difference between data changing in the source system and these changes being made available in the target tables or data warehouse.
- Data loading in Snowflake can be done in 3 ways.
 - Bulk Loading.
 - Near Real Time Loading.
 - Real Time Loading.
- Bulk loading of data is the process of loading source system data file in batches from cloud storage to tables in Snowflake at regular intervals of time and is scheduled.
- Bulk loading of data generally has a data latency of 15 mins and is the most common type of data loading.
- Advantages of bulk loading are
 - Simplicity.
 - Lower cost.
 - Supports multiple file formats.
 - Customization of load is possible.
- Near Real Time Loading of data refers to the process of frequently ingesting data into tables with low latency which is generally a few mins.
- There is no scheduling for Near Real Time data loads, it is event driven and on the occurrence of the event which is the arrival of the file on s3.

Summary

- Real Time loading of data ensures that the data is available for querying and analysis almost instantaneously after any changes occur in the source. The latency for Real Time data is just a few seconds.
- Real Time loading generally has higher cost as compared to Bulk loading and Near Real time loading.
- Real Time loading in Snowflake is enabled with Snowflake connector for Kafka and supports Avro and JSON format.
- The COPY can be used to load data from Snowflake stages(External or Internal) to tables in Snowflake. It can also be used to unload data from Snowflake tables to stages on AWS or internal stages.
- By default the COPY statement loads all files from the stage to the target table.
- By default any file that is loaded to a target table will not be re-loaded even if the COPY statement is re-run.
- Default COPY behavior can be changed using COPY options.
- A COPY option is an instruction that is added to the COPY statement to alter the default behavior of the COPY.
- COPY option **FORCE=TRUE** reloads the file even if the file is loaded to the target table.
- Column names can be provided in the COPY statement to load a subset of columns in the table.
- SELECT statement can be used in the FROM clause of the COPY statement.

Summary

- COPY option **FILES** can be used to load a specific file to the target table.
- COPY option **PATTERN** can be used to load files that match a pattern to the target table.
- COPY option **PURGE** deletes the files from the s3 after the COPY is executed.
- Copy option **TRUNCATECOLUMNS**, truncates columns data where the data length is greater than the length of the column, this prevents load failure or record rejection.
- **ON_ERROR** takes values, **ABORT_STATEMENT**, **SKIP_FILE**, **SKIP_FILE_num**, **SKIP_FILE_%**, **CONTINUE**.
- **SKIP_FILE** skips the file if it encounters even a single error.
- **SKIP_FILE_num** skips the file if it encounters error which is equal to or greater than value of **num**.
- **SKIP_FILE_%** changes the error limit dynamically by calculating it based on the number of rows in the file and skips the file if the number of errors is greater than the calculated error limit.
- Use COPY option **RETURN_FAILED_ONLY=TRUE** when you want to see only the failed or partially loaded files in the output, It must be used in conjunction with **ON_ERROR=CONTINUE**.
- COPY option **SIZELIMIT** can be used to break down file loading into chunks of files by limiting the amount of data that will be loaded in a single COPY statement. At a minimum at least 1 file will be loaded to the target table if you use **SIZELIMIT** irrespective of the limit provided.
- Files can be checked for errors before loading by using **VALIDATION_MODE**.
- **VALIDATION_MODE** takes 3 values **RETURN_ALL_ERRORS**, **RETURN_ERRORS** and **RETURN_n_ROWS**.
- **RETURN_ALL_ERRORS** returns all errors from **all the files** and displays it on the screen.
- **RETURN_ERRORS** returns a subset of errors returned by **RETURN_ALL_ERRORS**.
- **RETURN_n_ROWS** displays n rows where **n** is an integer number and can be used to profile the data, if it encounters an error within **n** rows the statement will error out.

Summary

- The default file format associated with a stage can be overridden by using file format options or a file format name in the COPY statement.
- Data can be loaded to Snowflake from s3 without a stage. [AWS_KEY_ID](#) and [AWS_SECRET_KEY](#) must be provided in the COPY to gain access to the s3.
- Data can be transformed in the COPY statement before it is loaded to the target table.
- Transforming data can result in slowing down of the load as computations need to be done.
- Some projects choose to load data as is and then transform the data inside Snowflake.
- Failed rows can be re-processed by using the table function [VALIDATE](#) and providing the table. Name.
- Column [REJECTED_RECORD](#) has the rejected rows stored which can be re-processed.
- Snowsight provides an “[Add Data](#)” wizard which can be used to load data into a Snowflake table with files from your local computer. This is useful for non-technical users who want to load data into a Snowflake table for analysis or joining with other Snowflake data sets.
- Multiple files can be uploaded at once with the “[Add Data](#)” wizard however the max file size of a single file is limited 250 MB and the supported formats are CSV,JSON, ORC,AVRO and parquet.

Thank you



Learn2
CloudData

Powered by

www.learn2clouddata.com

Find Us

www.learn2clouddata.com

+01 234 567 89